

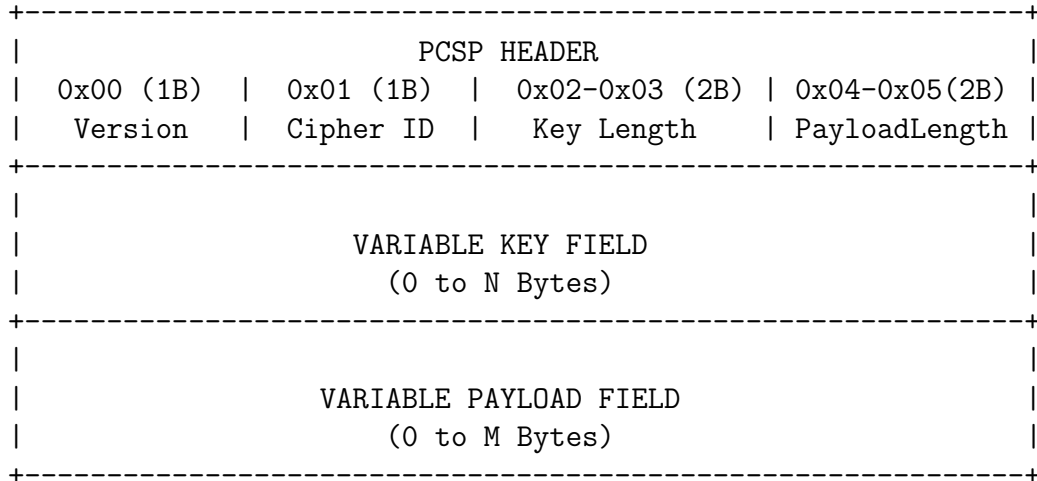
Project: Poly-Cipher Socket Protocol (PCSP)

Software Requirement Specifications)

Crypto-Study-Group, FCS

1 Introduction & System Architecture

The Poly-Cipher Socket Protocol (PCSP) is an educational application-layer protocol designed to demonstrate networking primitives and classical cryptographic transformations in C/C++. The system implements a synchronous, stateful Client-Server architecture over standard POSIX stream sockets.



2 Functional Requirements

2.1 Networking Architecture (NET)

- **REQ-NET-101 (Socket Binding):** The Server application shall instantiate a passive stream socket (SOCK_STREAM) using the AF_INET address family, binding to a configurable port and listening for incoming connections.
- **REQ-NET-102 (Connection Management):** The Server shall accept an incoming Client connection, process its packet completely, display the required diagnostic outputs, and cleanly close the connection before listening for the next client (Iterative Server Model).
- **REQ-NET-103 (Client Initiation):** The Client application shall establish an active connection to the Server via a specified IPv4 loopback or network address and port.

- **REQ-NET-104 (Byte Ordering):** All multi-byte integer fields transmitted across the socket (Key Length, Payload Length) must be normalized to Network Byte Order (Big-Endian) using `htons()` before transmission, and restored to Host Byte Order using `ntohs()` upon ingestion.

2.2 Protocol Framing & Parsing (PRT)

- **REQ-PRT-201 (Header Anatomy):** Every transmitted message must begin with a rigid 6-byte binary header structured as follows:
 - **Byte 0: Protocol Version** (Fixed at 0x01)
 - **Byte 1: Cipher Identifier** (Hexadecimal value mapped to specific classical ciphers)
 - **Bytes 2–3: Key Length** (unsigned 16-bit integer denoting length of the inline key string/array)
 - **Bytes 4–5: Payload Length** (unsigned 16-bit integer denoting length of the raw ciphertext)
- **REQ-PRT-202 (Stream Assembly):** The transmission packet must be packed contiguously in memory as: [Header] [Key Vector] [Payload (Ciphertext)].
- **REQ-PRT-203 (Server Parsing State Machine):** The server must parse the stream in two distinct phases:
 1. Read exactly 6 bytes to extract and evaluate the header attributes.
 2. Dynamic allocation and read operations for Key Length bytes followed by Payload Length bytes.

2.3 Diagnostic Display Requirements (DSP)

- **REQ-DSP-301 (Client-Side Telemetry):** Prior to transmission, the Client must output to standard console: the plaintext input, the selected Cipher ID, the Key metadata, and the resulting ciphertext rendered as a raw hexadecimal byte array.
- **REQ-DSP-302 (Server-Side Telemetry):** Upon packet receipt, the Server must print:
 1. The raw, unresolved hexadecimal stream arriving from the socket.
 2. The isolated ciphertext block (in hex format).
 3. The derived plaintext string obtained immediately post-decryption.

3 Cryptographic Subsystem Requirements (CIP)

The cryptographic subsystem must isolate each classical cipher within unified operational signatures:

$$\text{Encrypt}(P, K) \rightarrow C \quad \text{and} \quad \text{Decrypt}(C, K) \rightarrow P$$

Where transformations operate explicitly over ASCII alphabet bounding spaces or full $\mathbb{Z}_{256}$ byte domains.

3.1 Monoalphabetic Substitution Ciphers

- **REQ-CIP-401 (Caesar Cipher):**
 - **ID:** 0x10
 - **Specification:** Shifts alphabet characters by a fixed numeric key $K \in \mathbb{Z}_{26}$.
 - **Formula:** $C_i = (P_i + K) \pmod{26}$
- **REQ-CIP-402 (Affine Cipher):**
 - **ID:** 0x11
 - **Specification:** A two-part key $K = (a, b)$ where $\gcd(a, 26) = 1$. Decryption requires computing the modular multiplicative inverse $a^{-1} \pmod{26}$.
 - **Formula:** $C_i = (a \cdot P_i + b) \pmod{26}$
- **REQ-CIP-403 (Atbash Cipher):**
 - **ID:** 0x12
 - **Specification:** Keyless substitution mapping an alphabet to its exact reverse.
 - **Formula:** $C_i = 25 - P_i$
- **REQ-CIP-404 (Monoalphabetic Substitution):**
 - **ID:** 0x13
 - **Specification:** The key is a complete randomized 26-character alphabet array mapping explicitly to standard A-Z.

3.2 Polyalphabetic Substitution Ciphers

- **REQ-CIP-501 (Vigenère Cipher):**
 - **ID:** 0x20
 - **Specification:** Uses a variable-length string key K . The key shifts characters cyclically.
 - **Formula:** $C_i = (P_i + K_{i \pmod{\text{len}(K)}}) \pmod{26}$
- **REQ-CIP-502 (Beaufort Cipher):**
 - **ID:** 0x21
 - **Specification:** A variant of Vigenère where subtraction controls the mechanism, making encryption and decryption identical operations.
 - **Formula:** $C_i = (K_{i \pmod{\text{len}(K)}} - P_i) \pmod{26}$
- **REQ-CIP-503 (Autokey Cipher):**
 - **ID:** 0x22
 - **Specification:** Eradicates periodic key vulnerabilities by appending the plaintext itself to the priming key string.
 - **Formula:** $K_{\text{extended}} = K_{\text{prime}} \parallel P$

3.3 Polygraphic Substitution Ciphers

- **REQ-CIP-601 (Playfair Cipher):**
 - **ID:** 0x30
 - **Specification:** Substitutes pairs of letters (digrams) utilizing a dynamic 5×5 matrix generated from a keyword string. Handles matrix alignment adjustments for intersecting rows, columns, and blocks.
- **REQ-CIP-602 (Hill Cipher):**
 - **ID:** 0x31
 - **Specification:** Groups text into vectors of size n and multiplies them by an $n \times n$ invertible key matrix M over $\mathbb{Z}_{26}$. Decryption requires calculating the modular determinant and adjugate matrix to yield $M^{-1} \pmod{26}$.
 - **Formula:** $C = MP \pmod{26}$

3.4 Transposition Ciphers

- **REQ-CIP-701 (Rail Fence Cipher):**
 - **ID:** 0x40
 - **Specification:** Plaintext is written diagonally across a designated number of "rails" (the numeric key) and read off sequentially row-by-row.
- **REQ-CIP-702 (Columnar Transposition):**
 - **ID:** 0x41
 - **Specification:** Text is read horizontally into a matrix whose width matches a keyword length. The columns are then read out vertically in alphabetical order of the keyword characters.

4 Non-Functional Requirements & Safety (NFR)

- **REQ-NFR-801 (Memory Constraints):** The engine must explicitly avoid dynamic stack overflows by using bounds-checked array allocations (`strncpy`, `memcpy`) or standard container types (`std::vector`, `std::string` in C++).
- **REQ-NFR-802 (Graceful Error Recovery):** If the Server detects an unsupported Cipher ID (e.g., 0x99), an un-invertible Hill matrix, or a message payload length mismatch, it must reject the operation, print an explicit diagnostic warning code to standard output, and safely reset the socket channel without terminating the execution of the server process.